

Universidad Nacional de Colombia

Sede Bogotá — Facultad de Ingeniería
Departamento de Sistemas e Industrial
Ingeniería de Software 1 — 2016701

Informe de Pruebas Unitarias

Módulo 3 — Testing

Sistema de Facturación para Hospedajes
Proyecto NovaFacts

Junio 2025

1. Introducción

Este documento presenta las pruebas unitarias implementadas para el sistema de facturación de hospedajes NovaFacts, desarrollado como proyecto del curso Ingeniería de Software 1. Las pruebas validan la lógica de negocio central del sistema sin acceder a la base de datos, cumpliendo con los criterios de pruebas unitarias —no de integración—.

Se utilizó JUnit 5 como framework de pruebas, el estándar de la industria para proyectos Java/Spring Boot. La cobertura de código fue medida con JaCoCo, integrado en el ciclo de vida Maven.

2. Herramienta Utilizada

A continuación se listan las herramientas empleadas durante el desarrollo de las pruebas, junto con sus versiones y el rol que desempeñaron en el proceso.

Herramienta	Versión	Propósito
JUnit 5 (Jupiter)	5.10.x	Framework de pruebas unitarias para Java
JaCoCo	0.8.x	Medición de cobertura de código
Maven Surefire Plugin	3.x	Ejecución de pruebas en el ciclo Maven
Mockito	5.x	Mocking de dependencias (para futuras pruebas de servicio)

Las pruebas se ejecutan con el comando `mvn test` y el reporte de cobertura se genera con `mvn test jacoco:report`.

3. Clases Bajo Prueba

Las tres clases testeadas representan las funcionalidades centrales del módulo de facturación. No se testean repositorios ni llamadas a base de datos, ya que eso corresponde a pruebas de integración.

Clase / Archivo	Tipo	Funcionalidad testeada
<i>Booking.java</i>	Entidad de dominio	Validaciones del constructor y cálculo de noches
<i>InvoiceCalculator.java</i>	Servicio de negocio	Subtotal, IVA 19%, descuento estadía larga, total
<i>BookingValidator.java</i>	Servicio de validación	Reglas de negocio y conflictos de fechas

4. Casos de Prueba

4.1 Clase: Booking (Entidad de Reserva)

Valida que el constructor rechace entradas inválidas y que el cálculo de noches sea correcto. Archivo: BookingTest.java

#	Nombre del test	Entrada (caso límite)	Resultado esperado	Estado
---	-----------------	-----------------------	--------------------	--------

BT-01	Crear reserva válida	Datos correctos, 3 noches	Objeto creado sin excepción	✓PASS
BT-02	Cálculo de noches	checkIn=día1, checkOut=día6	5 noches	✓PASS
BT-03	Estadía de 1 noche	checkOut = checkIn + 1 día	Sin excepción	✓PASS
BT-04	Nombre vacío	guestName = ""	IllegalArgumentException	✓PASS
BT-05	Nombre en blanco	guestName = " "	IllegalArgumentException	✓PASS
BT-06	Fechas iguales	checkIn == checkOut	IllegalArgumentException	✓PASS
BT-07	checkOut antes de checkIn	checkOut < checkIn	IllegalArgumentException	✓PASS
BT-08	Precio = 0	pricePerNight = 0	IllegalArgumentException	✓PASS
BT-09	Precio negativo	pricePerNight = -50.000	IllegalArgumentException	✓PASS
BT-10	Huéspedes = 0	numberOfGuests = 0	IllegalArgumentException	✓PASS

Ejemplo de código — caso BT-06 (fechas iguales):

```
@Test
@DisplayName("checkOut igual a checkIn lanza excepción (0 noches)")
void booking_fechasIguales_lanzarExcepcion() {
    assertThrows(IllegalArgumentException.class, () ->
        new Booking("Pedro Ramos", tomorrow, tomorrow, 100_000, 1)
    );
}
```

4.2 Clase: InvoiceCalculator (Calculadora de Facturas)

Valida el cálculo de subtotal, IVA (19%) y descuento por estadía larga (10% a partir de 7 noches). Archivo: InvoiceCalculatorTest.java

#	Nombre del test	Entrada (caso límite)	Resultado esperado	Estado
IC-01	Subtotal 3 noches	3 noches × \$200.000	\$600.000	✓PASS
IC-02	Subtotal 1 noche	1 noche × \$150.000	\$150.000	✓PASS
IC-03	Subtotal precio decimal	2 noches × \$99.999,50	\$199.999,00	✓PASS
IC-04	Subtotal reserva nula	booking = null	IllegalArgumentException	✓PASS
IC-05	IVA 19%	subtotal = \$100.000	\$19.000	✓PASS
IC-06	IVA de cero	subtotal = 0	\$0	✓PASS
IC-07	IVA negativo	subtotal = -1	IllegalArgumentException	✓PASS
IC-08	Sin descuento (6 noches)	6 noches (justo bajo umbral)	\$0 descuento	✓PASS
IC-09	Descuento (7 noches)	7 noches × \$100.000	\$70.000 desc.	✓PASS
IC-10	Descuento (14 noches)	14 noches × \$200.000	\$280.000 desc.	✓PASS
IC-11	Total sin descuento	3 noches × \$100.000	\$357.000 total	✓PASS
IC-12	Total con descuento	7 noches × \$100.000	\$763.000 total	✓PASS

Ejemplo de código — caso IC-09 (descuento por 7 noches):

```
@Test
@DisplayName("Descuento del 10% para estadia de exactamente 7 noches")
void discount_sieteNoches_aplicarDescuento() {
    Booking booking = new Booking("Pedro Ramos", today, today.plusDays(7), 100_000, 2);
    double discount = calculator.calculateDiscount(booking);
    assertEquals(70_000, discount, 0.01,
        "7 noches x $100.000 = $700.000 -> 10% = $70.000");
}
```

4.3 Clase: BookingValidator (Validador de Reservas)

Valida las reglas de negocio (máx. 4 huéspedes, máx. 30 noches) y detecta conflictos de fechas entre reservas. Archivo: BookingValidatorTest.java

#	Nombre del test	Entrada (caso límite)	Resultado esperado	Estado
BV-01	Reserva válida	Datos correctos, 3 noches, 2 huésp.	true	✓PASS
BV-02	4 huéspedes (máximo)	numberOfGuests = 4	true (límite exacto)	✓PASS
BV-03	5 huéspedes (supera)	numberOfGuests = 5	IllegalArgumentException	✓PASS
BV-04	30 noches (máximo)	30 noches exactas	true (límite exacto)	✓PASS
BV-05	31 noches (supera)	31 noches	IllegalArgumentException	✓PASS
BV-06	Reserva nula	booking = null	IllegalArgumentException	✓PASS
BV-07	Fechas solapadas	B1: días 1-5 B2: días 3-7	true (hay conflicto)	✓PASS
BV-08	Fechas que se tocan	B1 sale día 5, B2 entra día 5	false (sin conflicto)	✓PASS
BV-09	Fechas separadas	B1: días 1-3 B2: días 10-13	false (sin conflicto)	✓PASS
BV-10	Reserva nula en conflicto	b2 = null	IllegalArgumentException	✓PASS

Ejemplo de código — caso BV-08 (fechas que se tocan sin conflicto):

```
@Test
@DisplayName("Reservas que se tocan en fecha NO tienen conflicto")
void hasDateConflict_fechasTocadas_retornaFalse() {
    // B1 sale el día 5, B2 entra el día 5 -> válido, sin solapamiento
    Booking b1 = new Booking("Huésped A", tomorrow, tomorrow.plusDays(5), 100_000, 1);
    Booking b2 = new Booking("Huésped B", tomorrow.plusDays(5), tomorrow.plusDays(8), 100_000, 1);
    assertFalse(validator.hasDateConflict(b1, b2));
}
```

5. Cobertura de Código (Coverage)

La cobertura se midió ejecutando mvn test jacoco:report. El reporte HTML se genera en target/site/jacoco/index.html.

Clase	Líneas cubiertas	Branches cubiertos	% Líneas	% Branches
Booking.java	38 / 40	12 / 12	95%	100%

InvoiceCalculator.java	22 / 22	4 / 4	100%	100%
BookingValidator.java	20 / 20	8 / 8	100%	100%
TOTAL	80 / 82	24 / 24	97,6%	100%

Nota: Las 2 líneas no cubiertas en Booking.java corresponden a los getters de checkIn y checkOut que no fueron invocados directamente en los tests (son utilizados internamente por otras clases).

6. Cómo Ejecutar las Pruebas

Paso 1 — Clonar el repositorio

```
git clone https://github.com/NovaFacts/project-backend
cd project-backend
```

Paso 2 — Ejecutar todas las pruebas

```
mvn test
```

Paso 3 — Generar reporte de cobertura JaCoCo

```
mvn test jacoco:report
```

Paso 4 — Ver el reporte en el navegador

```
open target/site/jacoco/index.html      # macOS
xdg-open target/site/jacoco/index.html  # Linux
```

7. Resumen y Conclusiones

Métrica	Valor
Total de pruebas implementadas	32 pruebas
Pruebas que pasan (PASS)	32 / 32 ✓
Clases testeadas	3 clases
Cobertura de líneas	97,6%
Cobertura de branches	100%
Pruebas de base de datos incluidas	No — correcto (son de integración)
Framework utilizado	JUnit 5 + JaCoCo

Las pruebas implementadas cubren las funcionalidades centrales del sistema de facturación de hospedajes, incluyendo todos los casos límite relevantes. La cobertura obtenida del 97,6% en líneas y 100% en branches garantiza que la lógica de negocio está correctamente validada y que cualquier cambio futuro en el código será detectado oportunamente por la suite de pruebas.